Polymorphism and Abstract Class in Python

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What is Polymorphism?

Definition

- **Polymorphism** means **“many forms”**.
- In OOP, it allows different objects to be **treated through a common interface**, even if they behave differently.

Goal

- Write **generic, reusable,** and **extendable** code.
- Use the **same method name** across multiple classes with **different implementations**.

Benefits

- Clean and modular code.
- Simplifies maintenance.
- Enables extensibility (e.g., adding new subclasses without changing existing code).

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Polymorphism – Basic Example

```
class Animal:
  def speak(self) -> None:
       print("Generic sound")

class Dog(Animal):
  def speak(self) -> None:
       print("Woof!")

class Cat(Animal):
  def speak(self) -> None:
       print("Meow!")
```

```
def make_speak(animal: Animal) -> None:
       animal.speak()

make_speak(Cat()) # Output "Meow!"
make_speak(Dog()) # Output "Woof!"
```

Polymorphism

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What are Abstract Classes and Methods?

- An **abstract class** is a class that **cannot be instantiated directly**. It provides a **template** for other classes to implement certain methods.

- An **abstract method** is a method **declared but not implemented** in the abstract class. Subclasses **must override it** to be instantiable.

How to define an Abstract Class:

- import from **abc** module;
- inherit from **ABC** (Abstract Base Class);
- use the `@abstractmethod` decorator.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Abstract Classes and Methods – Syntax

Syntax:

```
from abc import ABC, abstractmethod

class AbstractClass(ABC):
  @abstractmethod
  def abstract_method(self) -> str:
      pass
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Why use an Abstract Class?

- Enforce a **common interface** among subclasses.
- Promote **consistency and code organization**.
- Helpful in **large systems** where structure is essential.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Abstract Class – Basic Example

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
  @abstractmethod
  def description(self) -> str:
      pass

class Car(Vehicle):
  def description(self) -> str:
       return "I am a car."

class Bicycle(Vehicle):
  def description(self) -> str:
       return "I am a bicycle."
```

Key Points

- Attempting to instantiate Vehicle() raises TypeError.
- Both Car and Bicycle must implement description() to be instantiated.
- Promotes **polymorphic use**: you can call .description() on any Vehicle subclass.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Abstract Class – Usage Pattern Example

```
def show_description(vehicle: Vehicle) -> None:
   print(vehicle.description())

show_description(Car()) # Output "I'm a car."
show_description(Bicycle()) # Output "I'm a bicycle."
```

Polymorphism

✓ Clean, extendable code through abstraction!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY